UNIVERSITY OF THE PEOPLE

CS 2301-01 - AY2027-T1

Unit 4 Assignment

Submitted by: Mubashir Ali

Instructor: Joseph Ego

Date: September 24, 2026

Virtual Memory, Demand Paging, and Page Replacement Algorithms

Introduction

Cloud applications often experience changing workloads. An application may require relatively little memory during normal operation but suddenly needs much more memory when the number of users, data requests, or computational tasks increases. Virtual memory allows an operating system to manage these changing requirements by providing processes with virtual address spaces while using physical memory more efficiently. Two important virtual memory techniques are demanding paging and page replacement. Demand paging brings pages into physical memory when they are needed, while page replacement determines which page should be removed when available physical frames are exhausted. These mechanisms can improve memory utilization, but they can also introduce performance costs when page faults become frequent.

1. How Demand Paging Works in Virtual Memory

Demand paging is a virtual memory technique in which pages are loaded into physical memory only when a process references them. Instead of requiring the entire application to be loaded into RAM at startup, the operating

system can keep inactive pages in secondary storage and bring them into physical memory when necessary (Arpaci-Dusseau & Arpaci-Dusseau, 2023; Hailperin, 2019).

The process begins when the CPU generates a virtual address. The Memory Management Unit (MMU) uses the page table to determine the physical frame corresponding to the requested virtual page. If the page is already resident in RAM, the address can be translated, and the memory operation continues. If the requested page is not in physical memory, a page of fault occurs. The operating system is located in a free frame. If no free frame exists, it selects a resident page for replacement. The required page is loaded from secondary storage, the page table is updated, and the interrupted operation is restarted (Hailperin, 2019; Silberschatz et al., 2018).

For example, consider a cloud application with a virtual address space requiring 2 GB while the available physical memory assigned to it is only 512 MB. The application does not necessarily access all 2 GB at the same time. With demand paging, only the pages currently needed can occupy physical memory. A rarely used component, such as an administrative reporting module, does not have to remain in RAM while the application is processing frequently accessed user requests.

Paging also divides virtual memory into fixed-size pages and physical memory into frames of the same size. This allows pages to be placed in different physical locations instead of requiring one contiguous region of RAM (Arpaci-Dusseau & Arpaci-Dusseau, 2023). OSTEP identifies the elimination of external fragmentation and the flexibility of sparse virtual address spaces as important advantages of paging.

Therefore, demand paging can increase the effective utilization of physical memory and allow multiple applications to coexist. However, its effectiveness depends on the application's memory-access pattern and the amount of physical memory available.

2. Impact of Demand Paging on System Performance

Demand paging can improve overall system utilization because physical memory is allocated to pages that are actively required. This is particularly useful in a cloud environment where multiple applications or virtual machines may compete for limited memory. Instead of reserving enough RAM for the complete virtual address space of every application, the system can keep only actively used portions of resident.

The major disadvantage is the cost of a page of fault. A normal memory access is much faster than an operation that requires the operating system to retrieve a page from secondary storage. Consequently, frequent page faults can significantly increase memory-access time and application response time (Hailperin, 2019; Silberschatz et al., 2018).

The application's working set is therefore important. If an application repeatedly accesses a relatively small group of pages, those pages can remain in physical memory, and the page-fault rate can remain low. For example, a web application may repeatedly access code, session information, and database buffers. Once these pages are resident, subsequent accesses can proceed without repeatedly loading them from storage.

In contrast, suppose a workload suddenly expands and begins accessing a much larger collection of pages than physical memory can hold. The system may repeatedly remove pages and then reload them shortly afterward. This can produce a high page-fault rate and eventually thrash, in which the system spends excessive time handling

memory transfers rather than executing useful application work. Silberschatz et al. (2018) specifically discuss thrashing as an important consequence of insufficient frames relative to an application's memory requirements.

Performance can therefore be evaluated using several metrics, including page-fault rate, response time, throughput, CPU utilization, and memory utilization. A lower page-fault rate generally reduces the overhead associated with disk or storage operations. For cloud providers, this is important because unexpected memory pressure can produce latency spikes and reduce the number of applications that a server can support efficiently.

Modern operating system implementations also use sophisticated memory-reclamation techniques. For example, the Linux kernel documentation explains that memory pressure can trigger page reclaim, and that reclaim may occur asynchronously or synchronously depending on system conditions. This illustrates that virtual-memory management is not simply moving pages whenever RAM becomes full; the operating system must balance memory availability with performance.

3. Comparison of Four Page Replacement Algorithms

When a page of fault occurs and all physical frames are occupied, a page replacement algorithm determines which resident page should be removed. Four useful algorithms for comparison are FIFO, LRU, Optimal, and Random.

First-In, First-Out (FIFO)

FIFO removes the page that has been in memory for the longest time. Its main advantage is simplicity because the operating system only needs to maintain the order in which pages entered memory.

However, FIFO does not consider how frequently a page is being accessed. A page that entered memory early may still be heavily used, yet FIFO can remove it simply because it is old. OSTEP demonstrates that FIFO can also experience Belady's anomaly, where increasing the number of frames can sometimes increase the number of page faults.

For the cloud application, FIFO has low management overhead but may produce unnecessary page faults when the workload has strong locality.

Least Recently Used (LRU)

LRU removes the page that has not been used for the longest period. Its basic assumption is that recently accessed pages are more likely to be accessed again soon.

LRU can perform well when an application exhibits temporal locality. For example, if a web service repeatedly accesses the same database buffers, LRU is likely to retain those active pages and remove pages that have not been accessed recently.

The disadvantage is that maintaining exact access history introduces overhead. Nevertheless, LRU is an important practical strategy because it uses information about actual memory behavior rather than simply considering when pages enter memory. OSTEP's page-replacement material includes LRU as a policy specifically designed to exploit locality.

Optimal Replacement

The Optimal algorithm replaces the page whose next reference occurs farthest in the future. If the operating system knew the complete future reference sequence, this strategy would produce the minimum possible number of page faults for that particular sequence.

Optimal is therefore extremely useful as a theoretical benchmark. However, an operating system cannot normally know exactly which pages an application will reference in the future. Consequently, it is not generally a practical replacement algorithm for a live cloud application (Silberschatz et al., 2018; Tanenbaum & Bos, 2015).

Random Replacement

Random replacement selects a page randomly when replacement is required. It has very low management overhead because it does not need detailed information about page age or access history.

Its weakness is that it ignores locality. It may remove a page that the application will immediately need. Therefore, its page-fault behavior can vary considerably depending on the workload. It is useful as a simple baseline, but it does not intentionally preserve frequently accessed pages.

Algorithm	Selection rule	Strength	Main limitation
FIFO	Oldest resident page	Simple and inexpensive	Can remove frequently used pages
LRU	Least recently used page	Exploits temporal locality	Requires access-history information
Optimal	Farthest future use	Theoretical minimum faults	Future references are unknown
Random	Random resident page	Very low overhead	Ignores locality

The actual page-fault rate depends on the reference pattern, number of available frames, and working-set size. Consequently, it would be inaccurate to claim that one practical algorithm will always produce the lowest page-fault rate for every cloud workload.

4. Evaluation and Recommendation

For the new application, LRU or a practical approximation of LRU would be a reasonable choice when workload measurements show strong temporal locality. It attempts to retain recently accessed pages and remove colder pages, which aligns with applications that repeatedly use a relatively stable working set.

This conclusion is also consistent with modern operating system practice. The Linux kernel documentation describes Multi-Gen LRU as an LRU-based approach designed to improve page reclaim under memory pressure by representing access to recency and exploiting locality. This is useful evidence that recency-based memory reclamation remains relevant in contemporary systems, although a production kernel implementation is considerably more sophisticated than textbook LRU.

FIFO could still be appropriate when implementation of simplicity is the dominant concern, while Random may be useful as a low-overhead baseline. Optimal should be used primarily for comparison and experimentation because it provides a theoretical lower bound on page faults rather than a realistic implementation strategy.

The cloud provider should ultimately validate the choice using measurements from the application's actual workload. Important metrics include page faults per second, average response time, throughput, CPU utilization, and memory pressure. If measurements show that page faults remain high despite the replacement policy, simply changing algorithms may not solve the problem. The application may instead require additional physical memory, workload changes, or better working-set management.

Source Evaluation

The sources used in this analysis were selected based on relevance, authority, currency, and potential bias. Arpaci-Dusseau and Arpaci-Dusseau's *Operating Systems: Three Easy Pieces* and Hailperin's *Operating Systems and Middleware* are highly relevant because they directly explain virtual memory, paging, page faults, and page replacement in an operating-systems context. Silberschatz et al.'s *Operating System Concepts* and Tanenbaum and Bos's *Modern Operating Systems* provide independent academic textbook perspectives, which helps avoid relying on a single author's treatment of the topic. MIT OpenCourseWare provides university-level educational material on virtual memory, while official Linux kernel documentation provides evidence about how modern operating systems handle memory reclamation. These sources are more authoritative for this assignment than general programming blogs or commercial tutorials. The textbooks are not all recent editions, but the fundamental mechanisms discussed remain central to modern virtual-memory systems. The Linux documentation provides a more current perspective on practical memory management implementation.

Conclusion

Demand paging allows a virtual-memory system to load pages only when they are needed, improving physical-memory utilization for applications with variable memory demands. Its major performance cost is the page fault, particularly when frequent faults cause excessive paging and thrashing. Page replacement algorithms determine which pages should be removed when physical memory is full.

FIFO provides simple implementation but ignores recent access behavior. LRU uses temporal locality and can perform well for workloads with repeated access patterns. Optimal provides the theoretical minimum number of page faults but cannot normally be implemented because future references are unknown. Random has low overhead but does not use information about locality.

For the cloud application, an LRU-based strategy or practical approximation should be considered when workload measurements demonstrate strong temporal locality. However, the final decision should be based on actual performance measurements rather than assuming that one algorithm is universally optimal. Combining demand paging with appropriate page replacement, sufficient physical memory, and continuous monitoring can help the cloud provider maintain efficient memory utilization while reducing latency and avoiding thrashing.

References

Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2023). *Operating systems: Three easy pieces* (Version 1.10). Arpaci-Dusseau Books. https://pages.cs.wisc.edu/~remzi/OSTEP/

Hailperin, M. (2019). *Operating systems and middleware: Supporting controlled interaction* (Rev. 1.3.1). Gustavus Adolphus College.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley. https://uat.store.wiley.com/en-us/operating-system-concepts-10th-edition-p-9781119320913R150

Tanenbaum, A. S., & Bos, H. J. (2015). *Modern operating systems* (4th ed.). Pearson. https://www.pearson.com/en-us/subject-catalog/p/Tanenbaum-Lecture-Power-Points-for-Modern-Operating-Systems-4th-Edition/P200000003311/9780133591620

Massachusetts Institute of Technology. (2009). *Virtual memory*. MIT OpenCourseWare, 6.004 Computation Structures. https://ocw.mit.edu/courses/6-004-computation-structures-spring-2009/resources/mit6_004s09_lec17/

The Linux Kernel Organization. (2026). *Concepts overview: Memory management*. Linux kernel documentation. https://docs.kernel.org/admin-guide/mm/concepts.html

The Linux Kernel Organization. (2026). *Multi-Gen LRU*. Linux kernel documentation. https://docs.kernel.org/mm/multigen_lru.html